Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: _____ **Carné**: _____

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una canalización CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado
Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:
GitHub: https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet
El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio
En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable

---

2. Revisar la aplicación:
Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>

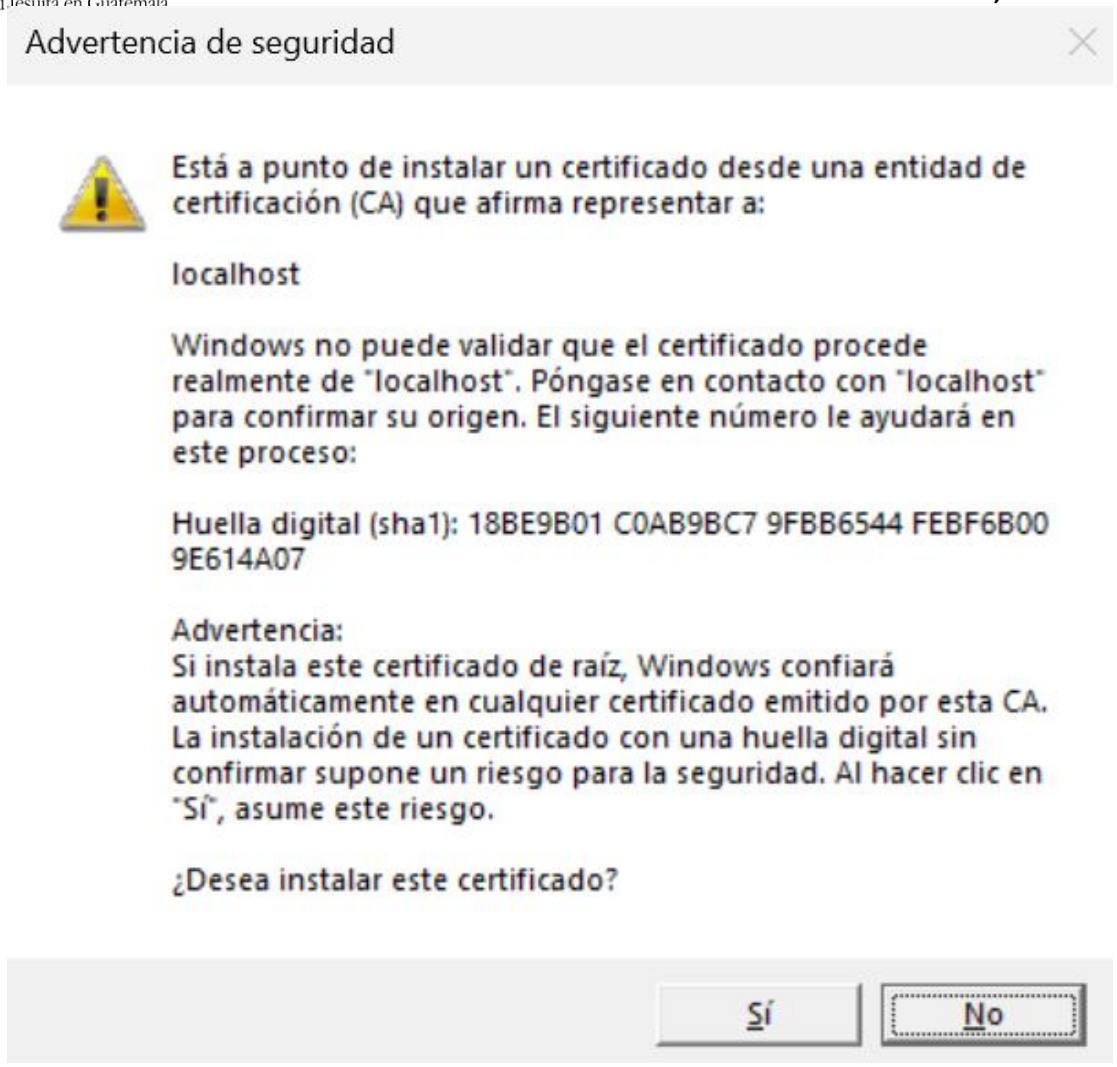


Figura 1. Paso 2: advertencia de seguridad de Windows al ejecutar la aplicación localmente (certificado de localhost)

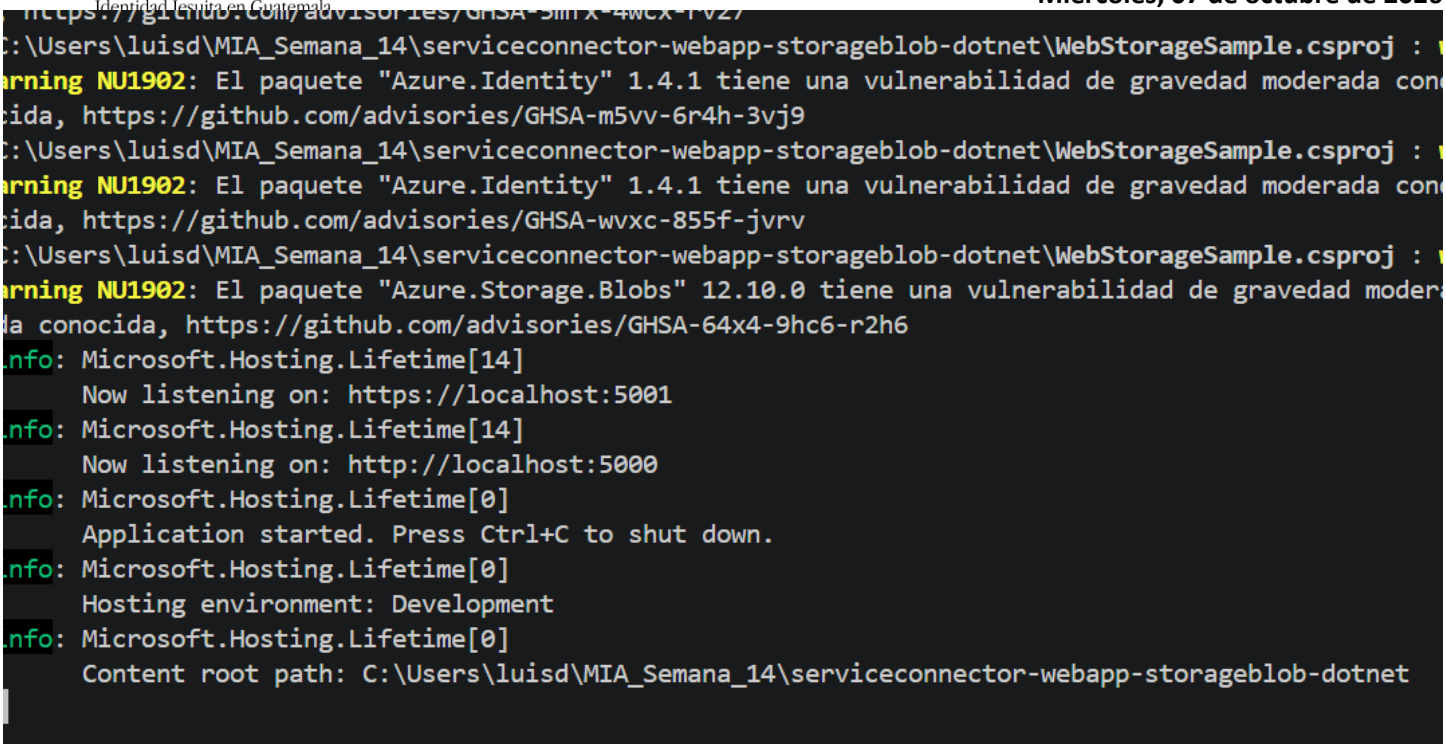


Figura 2. Paso 2: aplicación ejecutándose localmente con dotnet run (escuchando en localhost:5000 y 5001)

---

3. Crear DotNet Env

a) En terminal, instalar: **dotnet add package DotNetEnv**

b) Crear .env file en raíz de proyecto, valores ->

AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/

AZURE_STORAGE_CONTAINER_NAME=default

c) Crear .env.production con los mismos valores en el mismo lugar

d) Anadir cambio brindado en program.cs

e) dotnet clean -> dotnet restore -> dotnet build -> dotnet run

4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

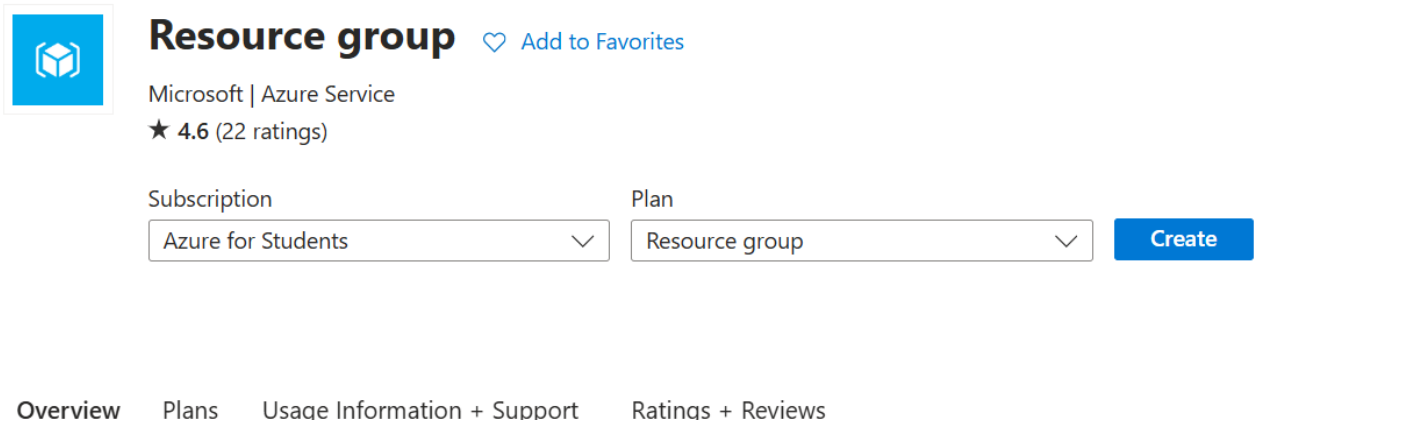


Figura 3. Paso 4: creación del Resource Group en Azure Portal (suscripción Azure for Students)

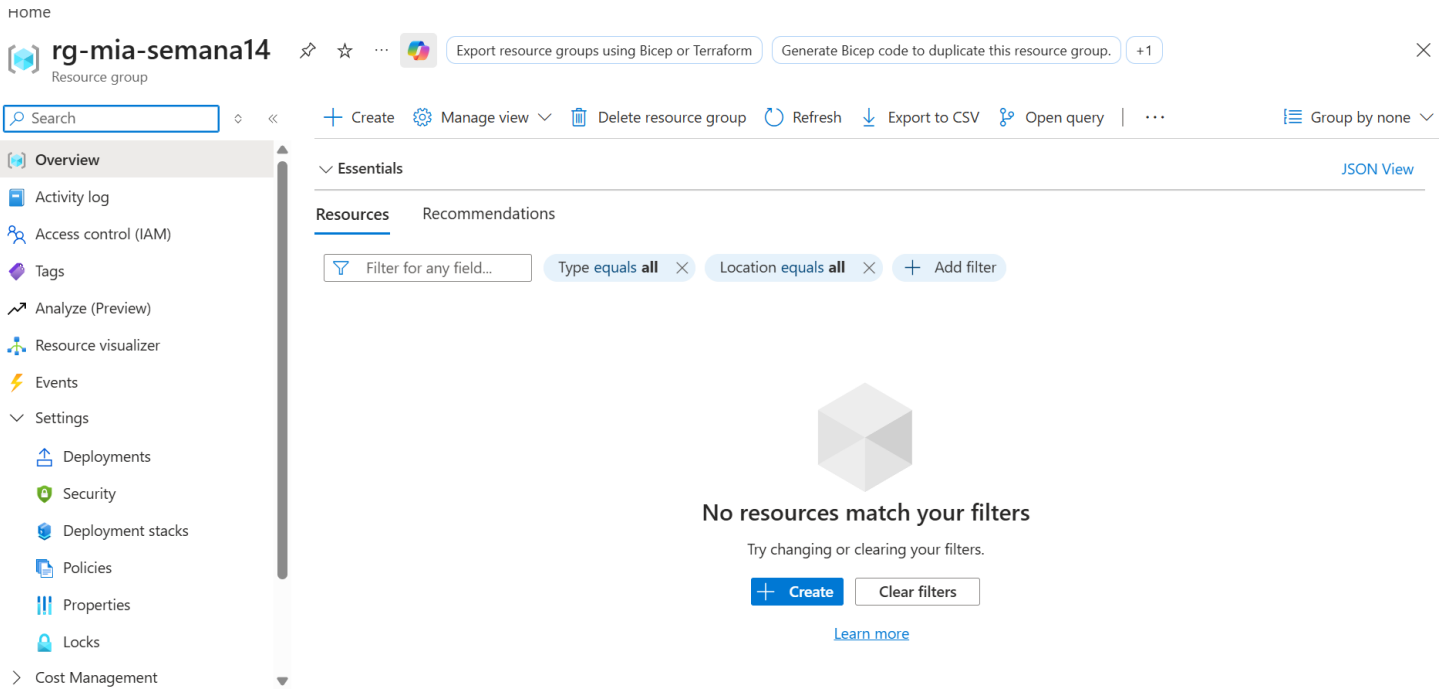


Figura 4. Paso 4: Resource Group rg-mia-semana14 creado

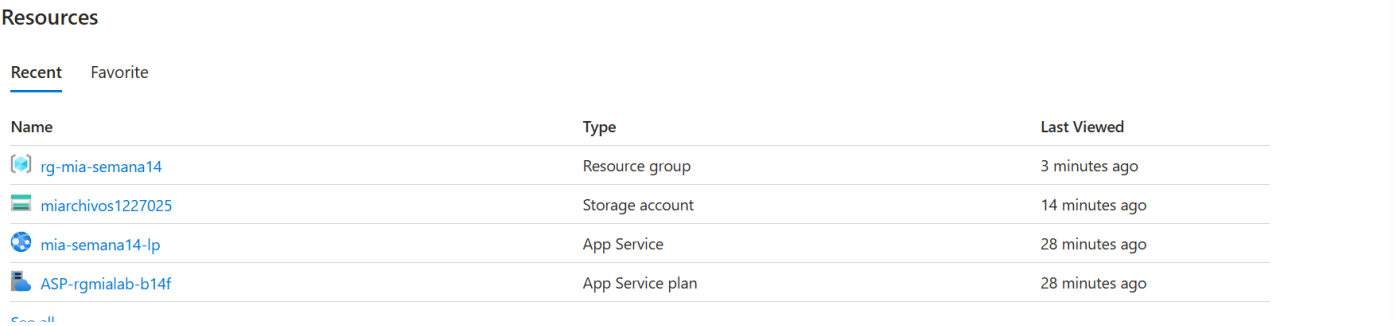


Figura 5. Paso 4: recursos creados: Storage Account, App Service y plan de App Service

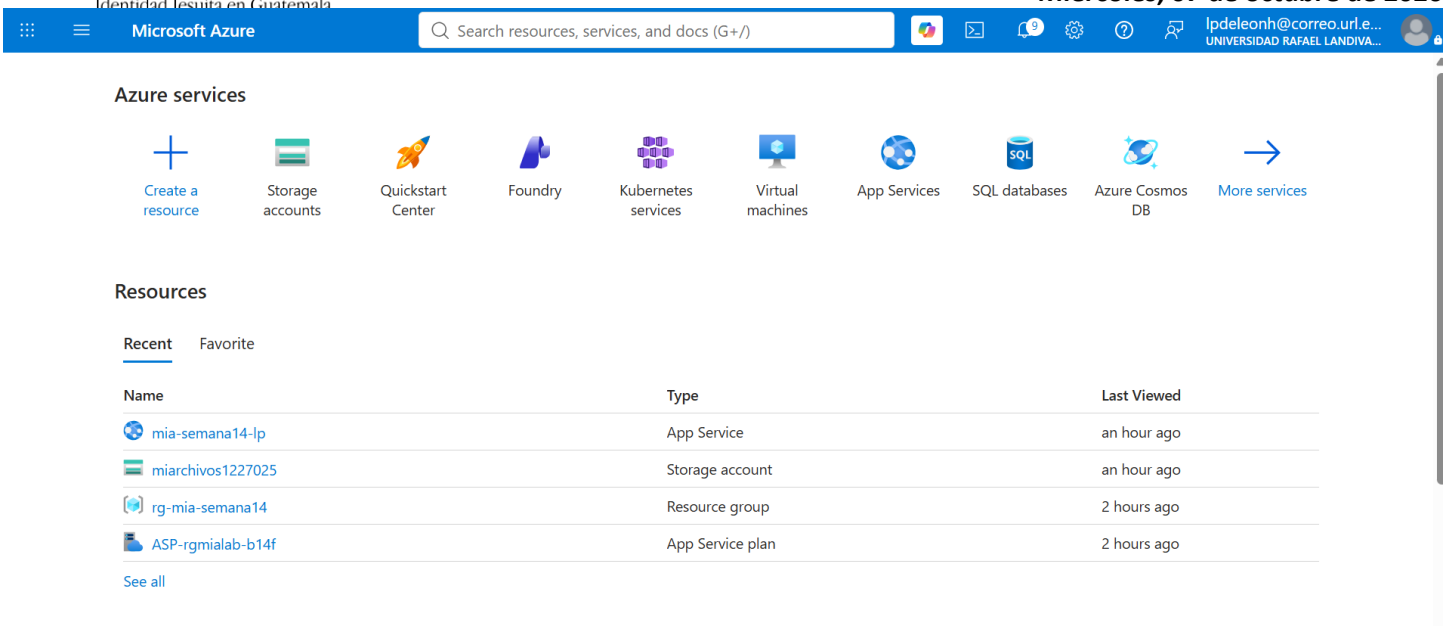


Figura 6. Paso 4: recursos recientes en el inicio de Azure Portal

5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

a) Azure Portal → **Storage accounts → tu Storage Account → Data storage → Containers**
b) Crear contenedor "default
c) Activa identidad administrada en App Service: **Azure Portal → App Service → Settings → Identity → System assigned: on → Save (web app creada semana 13)**
d) Configurar la conexión entre App Service y Blob Storage: **App Service → Settings → Service Connector → Create → Seleccionar "Storage Blob" → Seleccionar Storage Account [tu_blob_name] → Authentication "System-assigned managed identity" → Review + Create**.
e) Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → Settings → Endpoints → Blob service

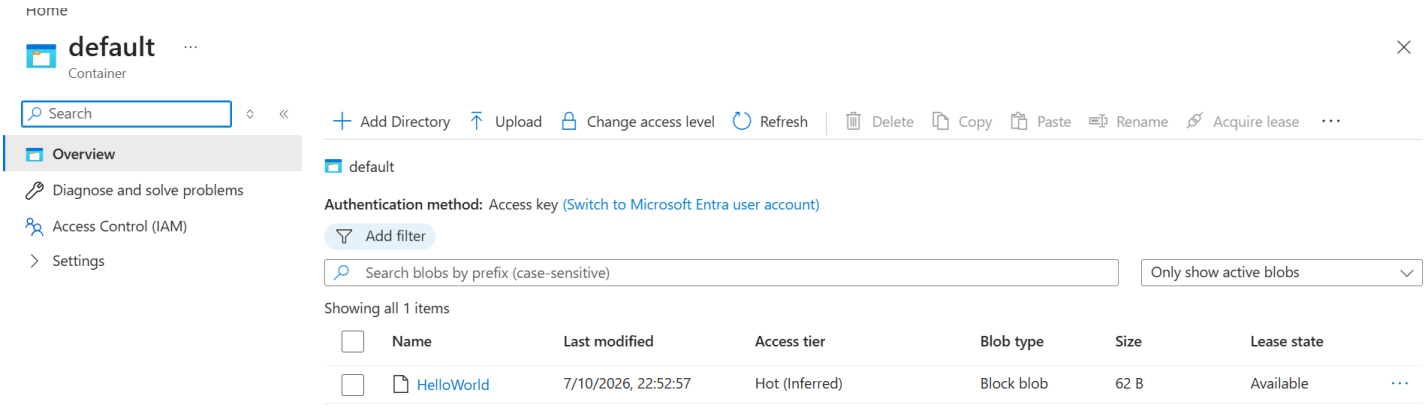


Figura 7. Paso 5b: contenedor default de la Storage Account con el archivo HelloWorld

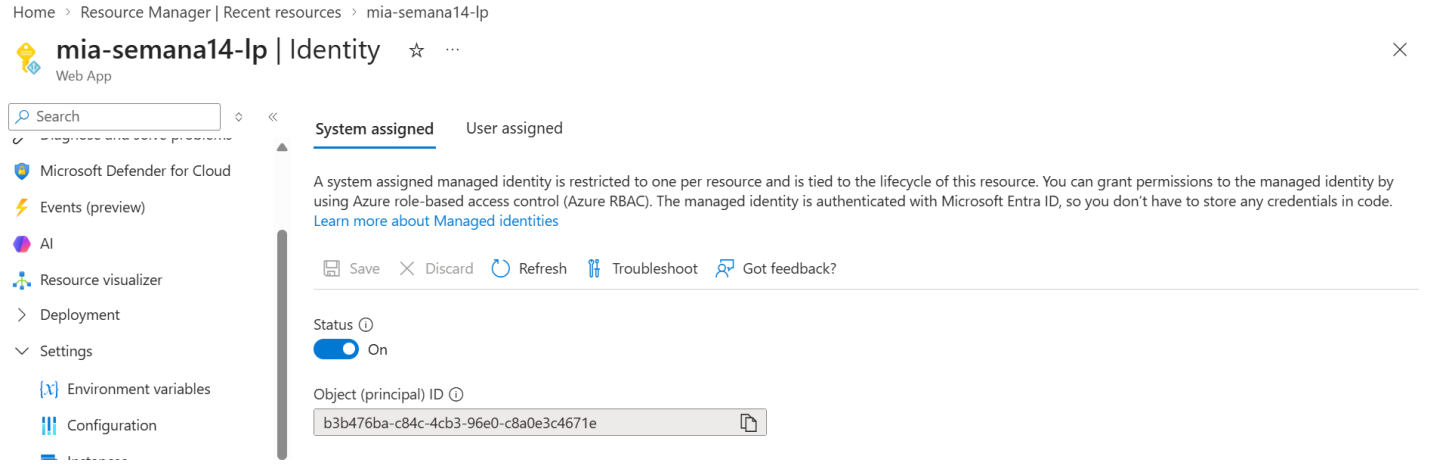


Figura 8. Paso 5c: identidad administrada asignada por el sistema activada (System assigned: On) en mia-semana14-lp

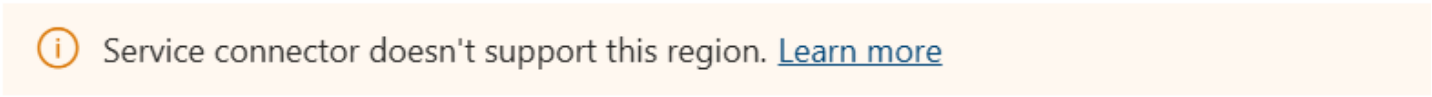


Figura 9. Paso 5d: aviso de Service Connector: la región no es compatible

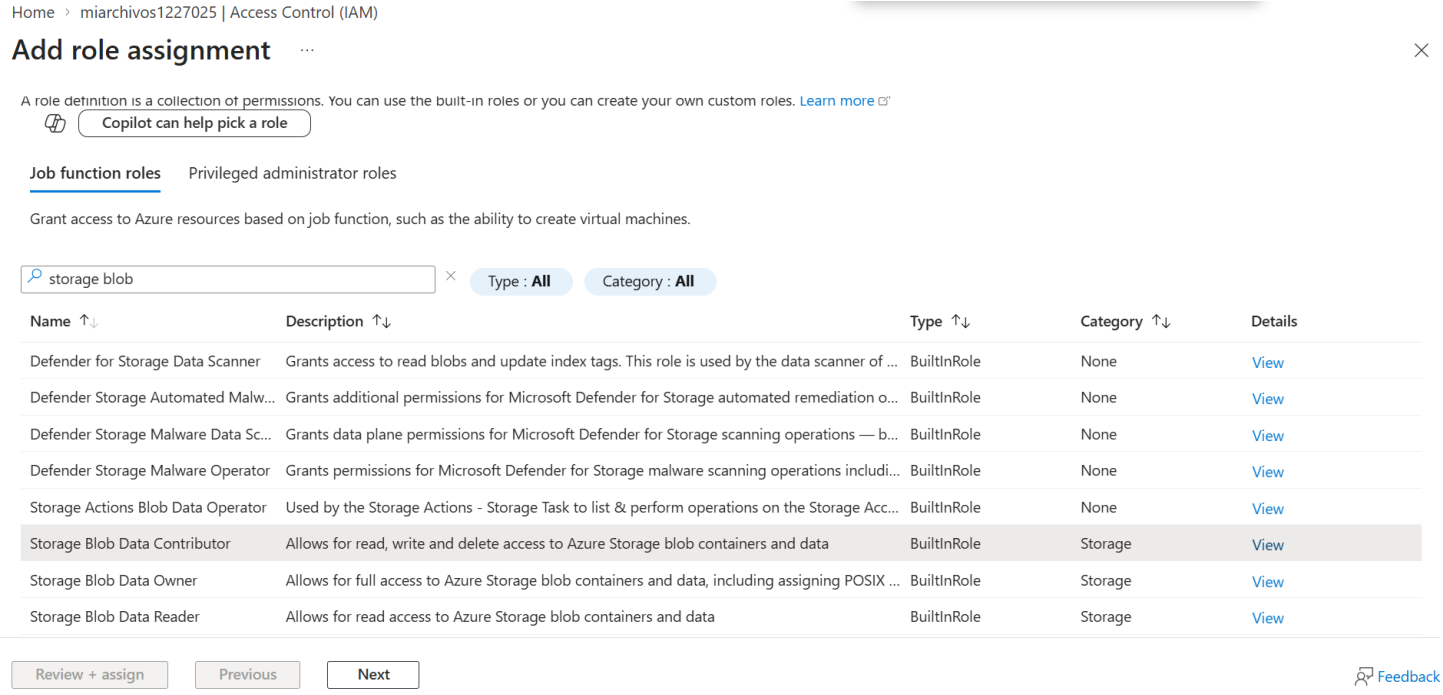


Figura 10. Paso 5: asignación del rol Storage Blob Data Contributor en Access Control (IAM) de la Storage Account

---

6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

a) Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: az --version
b) Inicie sesión en Azure: az login
c) Verifique la cuenta de Azure activa: az account show
d) Verifique que pueda obtener un token para Azure Storage: az account get-access-token --resource https://storage.azure.com/
e) Si se muestra un accessToken la autenticación esta correcta, ejecutar -> dotnet run

f) Si hay error en acceso, ejecutar en terminarl vs code: az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"

Obtener info: az account show --query "{subscriptionId:id, user:user.name}" -o table
az storage account show --name “tu_storage” --query "{id:id, resourceGroup:resourceGroup}" -o table

g) Cerrar process ctr +c y logear de nuevo: az logout y az login, Por ultimo dotnet run

Nota: no olvidar subir cambios a repo github fork

```
PS C:\Users\luisd\MIA_Semana_14\serviceconnector-webapp-storageblob-dotnet> az login
Select the account you want to log in with. For more information on login with Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136
```

Figura 11. Paso 6b: inicio de sesión en Azure con az login

```
Python location 'C:\Program Files\Microsoft SDKs\Azure\CLI2\python.exe'
Config directory 'C:\Users\luisd\.azure'
Extensions directory 'C:\Users\luisd\.azure\cliextensions'

Python (Windows) 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)]

Legal docs and information: aka.ms/AzureCliLegal


Your CLI is up-to-date.
PS C:\Users\luisd\MIA_Semana_14\serviceconnector-webapp-storageblob-dotnet> az account show
{
  "environmentName": "AzureCloud",
  "homeTenantId": "0f78549d-3eec-43af-b56a-6f7b042d6c9a",
  "id": "2ea7864a-1e1e-494c-b9e4-b09a6bda8724",
  "isDefault": true,
  "managedByTenants": [],
  "name": "Azure for Students",
  "state": "Enabled",
  "tenantDefaultDomain": "correo.url.edu.gt",
  "tenantDisplayName": "Universidad Rafael Landivar",
  "tenantId": "0f78549d-3eec-43af-b56a-6f7b042d6c9a",
  "user": {
    "name": "lpdeleonh@correo.url.edu.gt",
```

Figura 12. Paso 6c: cuenta de Azure activa con az account show

Stack Query Cookies Headers Routing

AggregateException: One or more errors occurred. (Azure CLI authentication failed due to an unknown error. "az" no se reconoce como un comando interno o externo, programa o archivo por lotes ejecutable.)

System.Threading.Tasks.Task.ThrowIfExceptional(bool includeTaskCanceledExceptions)
System.Threading.Tasks.Task.Wait(int millisecondsTimeout, CancellationToken cancellationToken)
System.Threading.Tasks.Task.Wait()
WebStorageSample.Pages.IndexModel.OnGet() in **Index.cshtml.cs**
26. `StorageHelper.UploadBlob(Environment.GetEnvironmentVariable(Const.ENDPOINT_ENV_KEY), Const.CONTAINER_NAME, Const.BLOB_NAME, content).Wait();`
Microsoft.AspNetCore.Mvc.RazorPages.Infrastructure.ExecutorFactory+VoidHandlerMethod.Execute(object receiver, object[] arguments)
Microsoft.AspNetCore.Mvc.RazorPages.Infrastructure.PageActionInvoker.InvokeHandlerMethodAsync()
Microsoft.AspNetCore.Mvc.RazorPages.Infrastructure.PageActionInvoker.InvokeNextPageFilterAsync()
Microsoft.AspNetCore.Mvc.RazorPages.Infrastructure.PageActionInvoker.Rethrow(PageHandlerExecutedContext context)
Microsoft.AspNetCore.Mvc.RazorPages.Infrastructure.PageActionInvoker.Next(ref State next, ref Scope scope, ref object state, ref bool isCompleted)
Microsoft.AspNetCore.Mvc.RazorPages.Infrastructure.PageActionInvoker.InvokeInnerFilterAsync()
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeNextResourceFilter>g__Awaited|25_0(ResourceInvoker invoker, Task lastTask, State next, Scope scope, object state, bool isCompleted)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.Rethrow(ResourceExecutedContextSealed context)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.Next(ref State next, ref Scope scope, ref object state, ref bool isCompleted)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.InvokeFilterPipelineAsync()
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)
Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.<InvokeAsync>g__Awaited|17_0(ResourceInvoker invoker, Task task, IDisposable scope)
Microsoft.AspNetCore.Authorization.AuthorizationMiddleware.Invoke(HttpContext context)
Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddlewareImpl.Invoke(HttpContext context)

Figura 13. Paso 6: error de autenticación de Azure CLI al subir un blob ("az" no se reconoce como comando)

WebStorageSample Home Privacy

Hello Service Connector! UTC Now: 08/10/2026 4:22:06 +00:00.

Learn about building Web apps with ASP.NET Core.

© 2021 - WebStorageSample - Privacy

Figura 14. Paso 6e: aplicación ejecutada localmente después de iniciar sesión (Hello Service Connector)

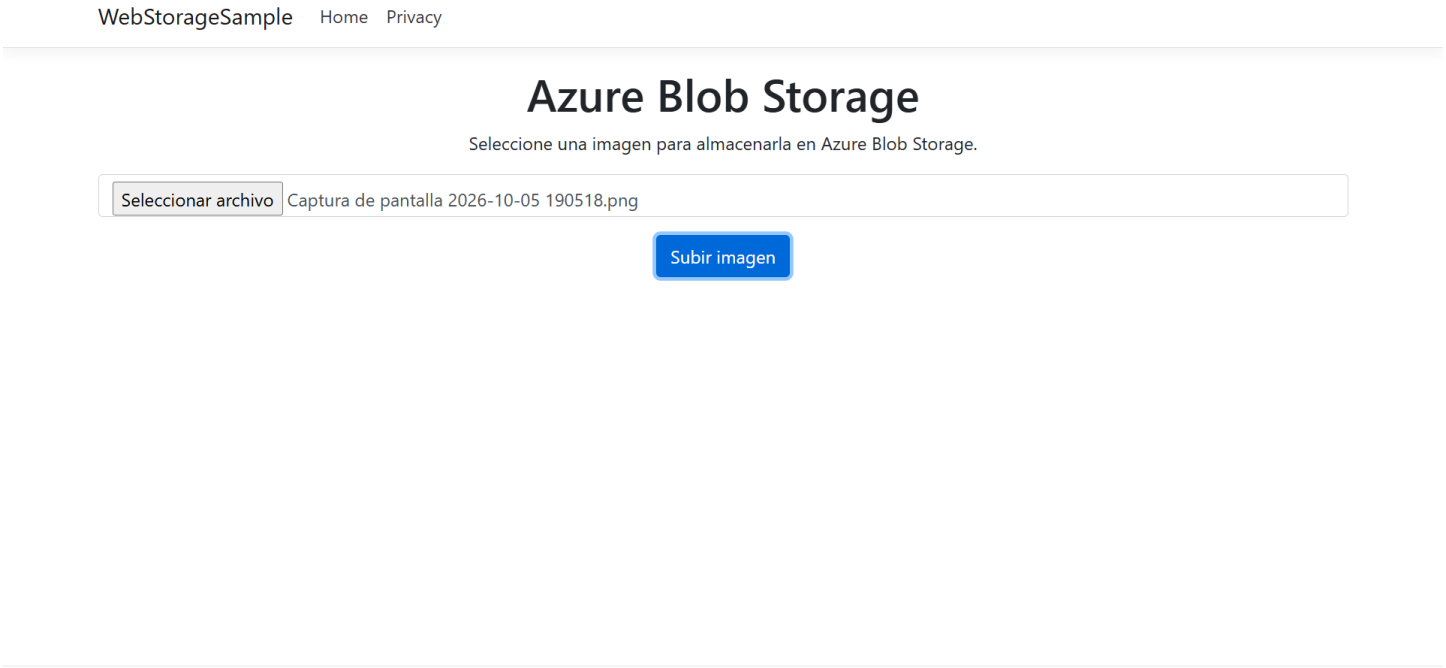


Figura 15. Paso 6: página de carga de la aplicación con un archivo seleccionado listo para subir

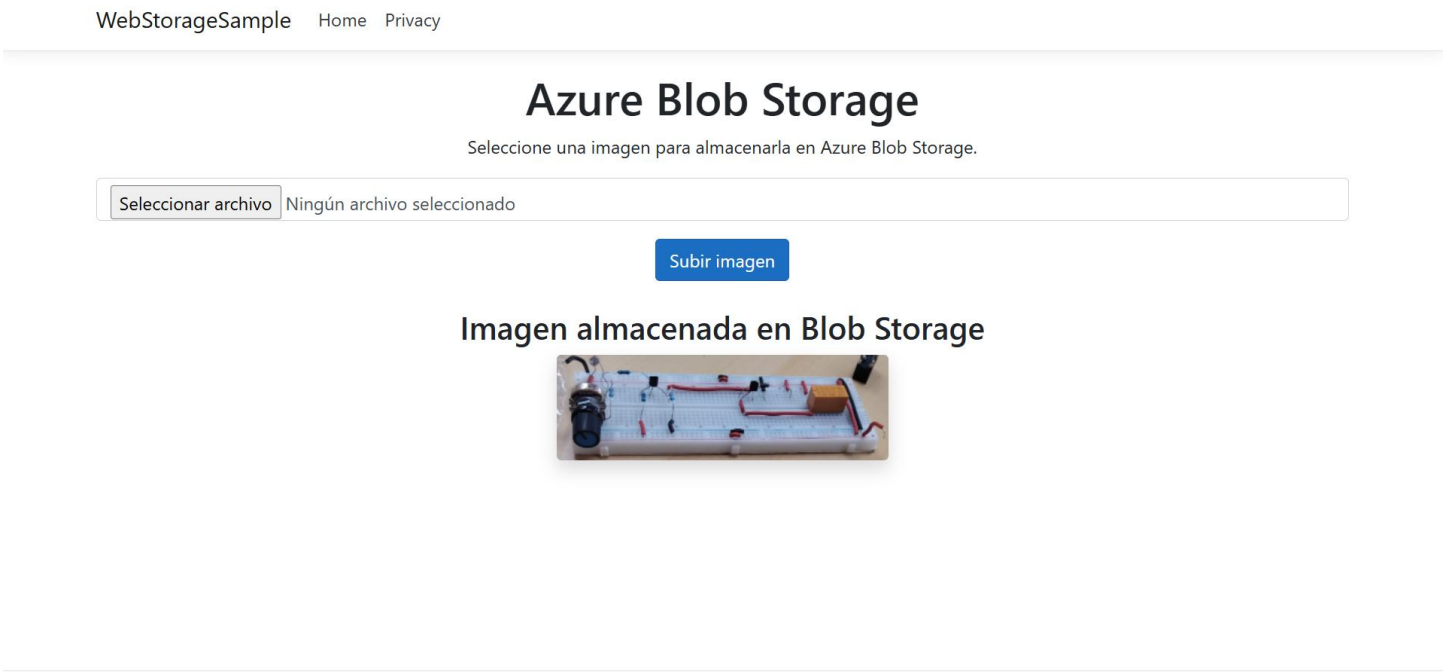


Figura 16. Paso 6: imagen almacenada en Blob Storage mostrada en la aplicación

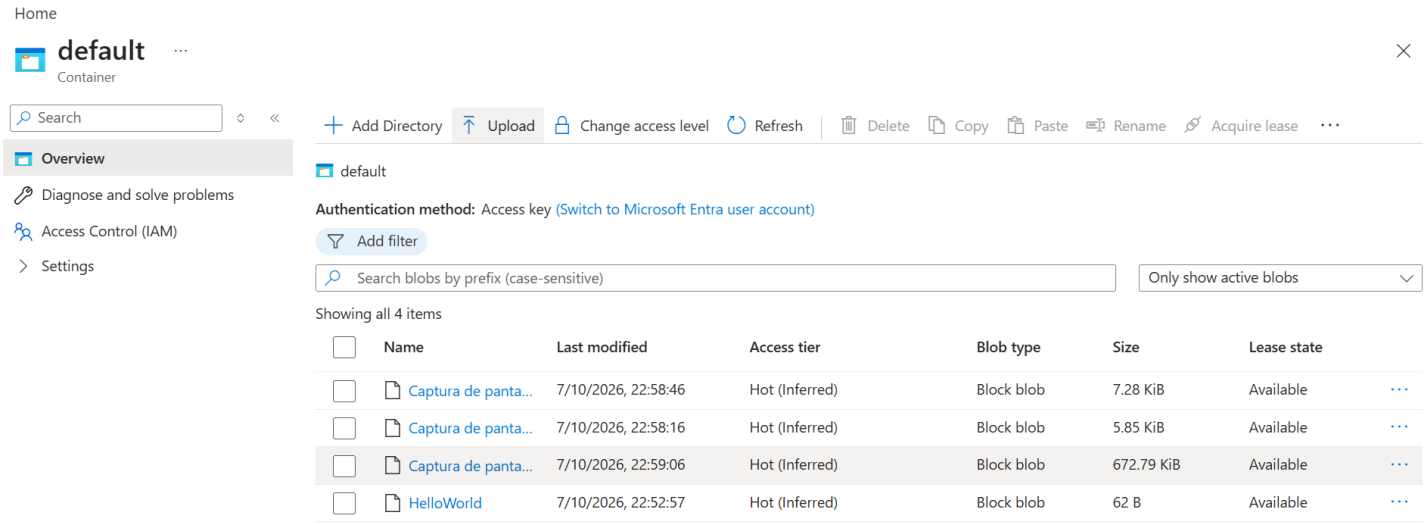


Figura 17. Paso 6: contenedor default con HelloWorld y tres capturas cargadas desde la aplicación



Figura 18. Paso 6 (nota): cambios subidos al repositorio de GitHub con el commit "Lab1 S14: prueba local con Blob Storage"

7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo “azure-pipelines.yml”
Colocar en el archivo lo siguiente:

```
trigger:
- main

pool:
  vmImage: 'windows-latest'

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '9.x'

  # Completar posteriormente
  azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'
  appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

# 1. Instalar .NET 9
- task: UseDotNet@2
  displayName: 'Instalar .NET 9 SDK'
  inputs:
    packageType: 'sdk'
    version: '$(dotnetVersion)'

# 2. Restaurar dependencias
- task: DotNetCoreCLI@2
  displayName: 'Restaurar dependencias'
  inputs:
    command: 'restore'
    projects: '**/*.csproj'

# 3. Configurar ambiente de producción
- powershell: |
    Copy-Item ".env.production" ".env" -Force
  displayName: 'Configurar .env de producción'

# 4. Compilar
- task: DotNetCoreCLI@2
  displayName: 'Compilar aplicación'
  inputs:
    command: 'build'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --no-restore'




# 5. Publicar
- task: DotNetCoreCLI@2
```

```
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'
    zipAfterPublish: true

# 6. Desplegar en Azure App Service
- task: AzureWebApp@1
  displayName: 'Desplegar en Azure App Service'
  inputs:
    azureSubscription: '$(azureSubscription)'
    appType: 'webApp'
    appName: '$(appName)'
    package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```

---

8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

a) Conectarse a Azure Devops son misma cuenta que azure portal -> https://aex.dev.azure.com/me

b) Crear pipeline: **En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml**

c) Probar seleccionando “run”

d) Conectar Azure DevOps con Azure: **Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre “MIA_Connection” -> Seleccionar “grant access permission to all pipelines” -> Save**

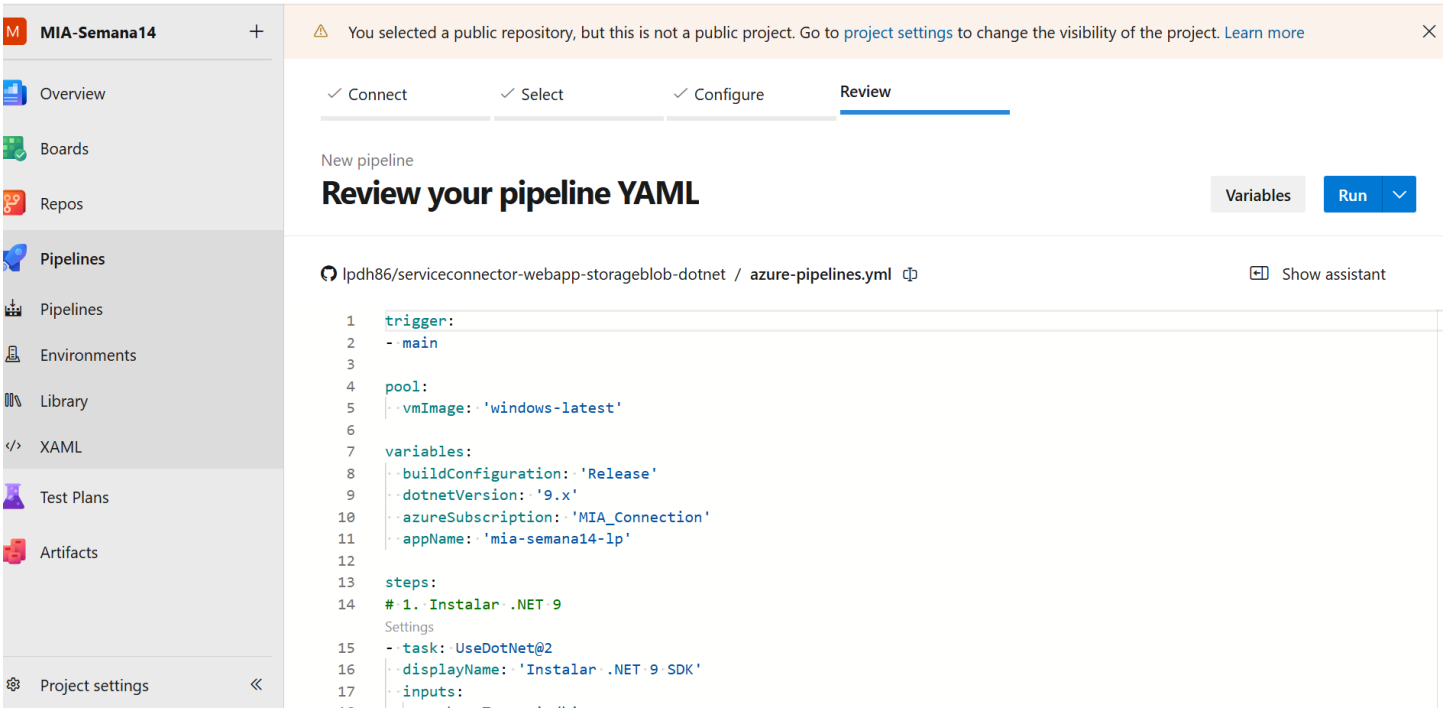


Figura 19. Paso 8b: revisión del YAML del pipeline (azure-pipelines.yml) en Azure DevOps

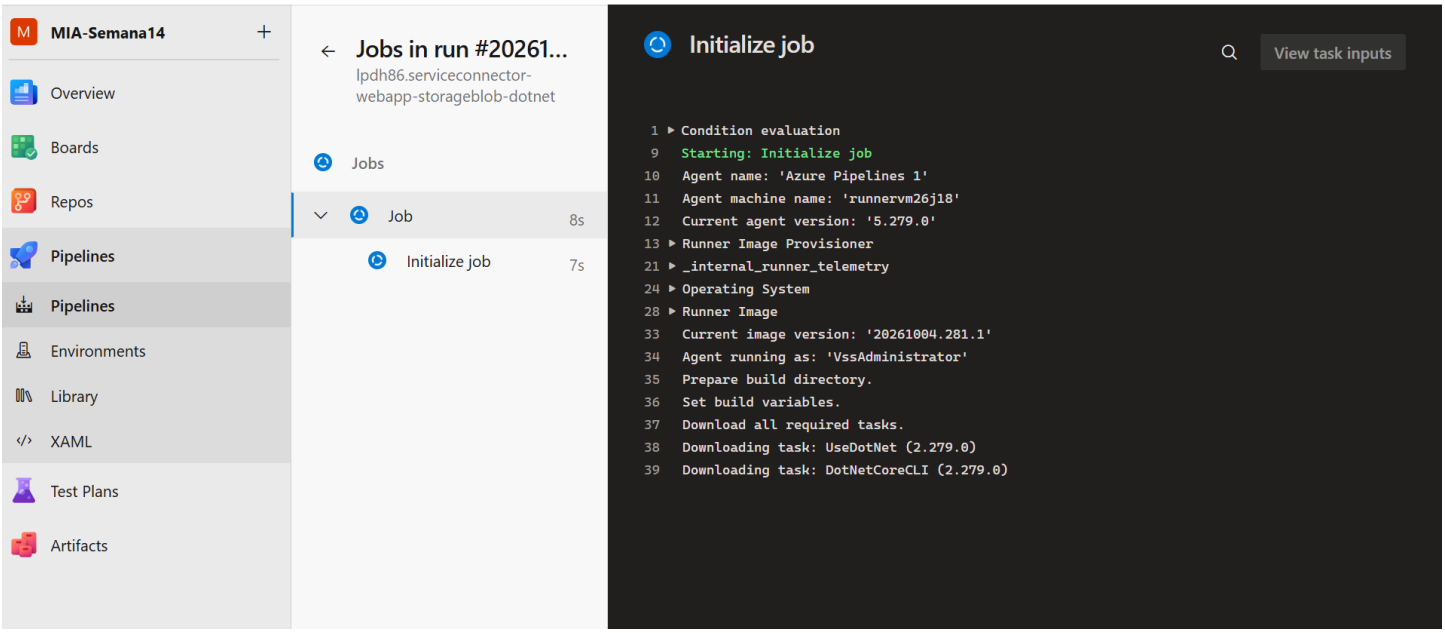


Figura 20. Paso 8c: ejecución del pipeline iniciando (Initialize job)

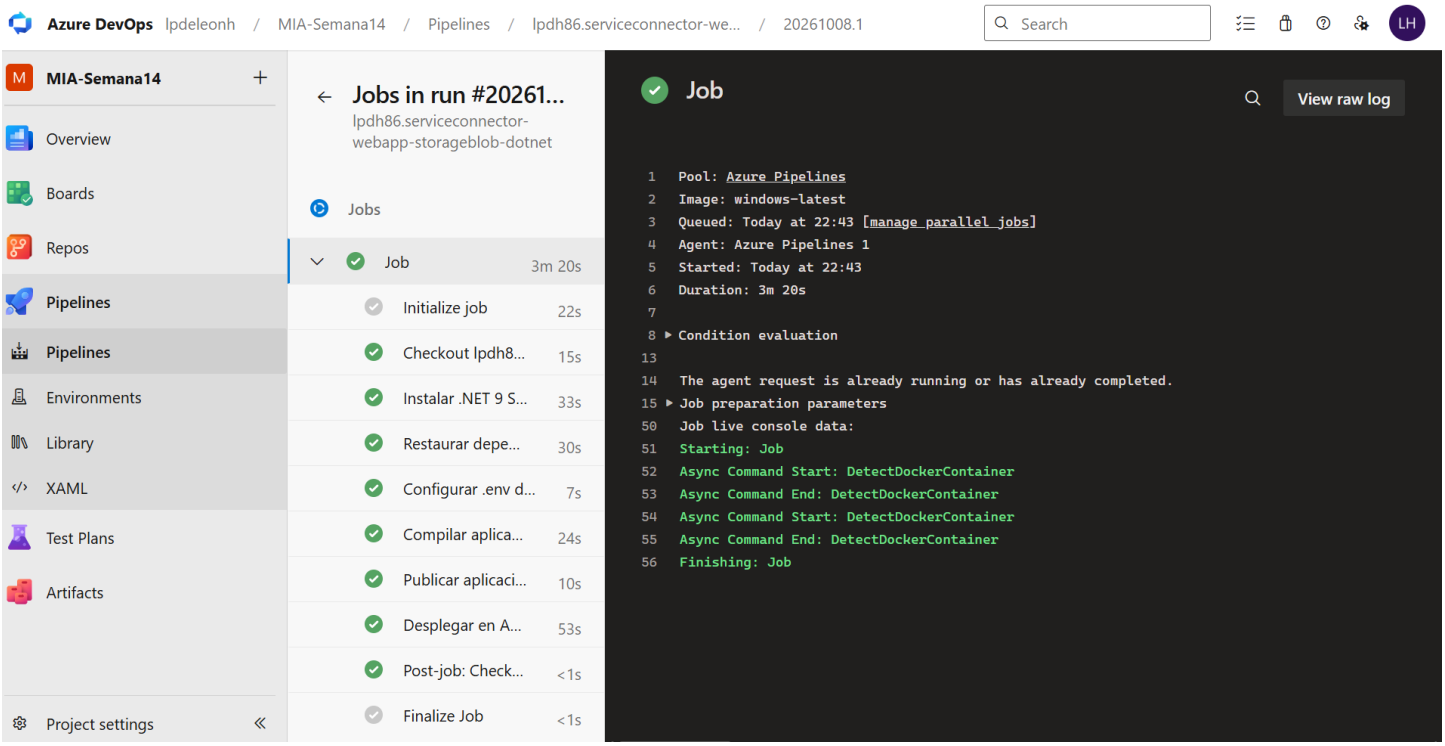


Figura 21. Paso 8c: ejecución del pipeline completada con éxito (restaurar, compilar, publicar y desplegar)

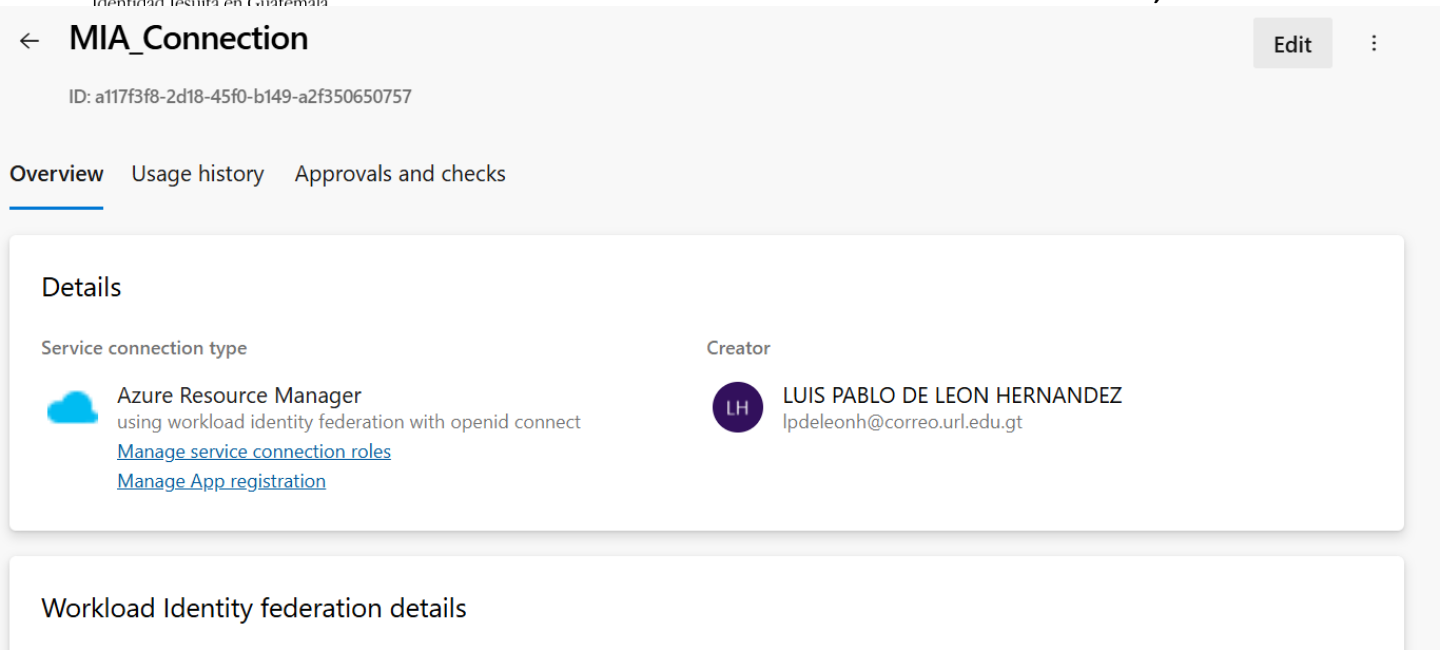


Figura 22. Paso 8d: service connection MIA_Connection (Azure Resource Manager con Workload Identity Federation)

---

8. Conectar Pipeline con App

a) Obtener nombre app: En Azure Portal App Services → tu aplicación → Copia nombre app

b) en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name
appName: Tu _app_name

c) Al hacer commit, verificar el pipeline se ejecuta automáticamente

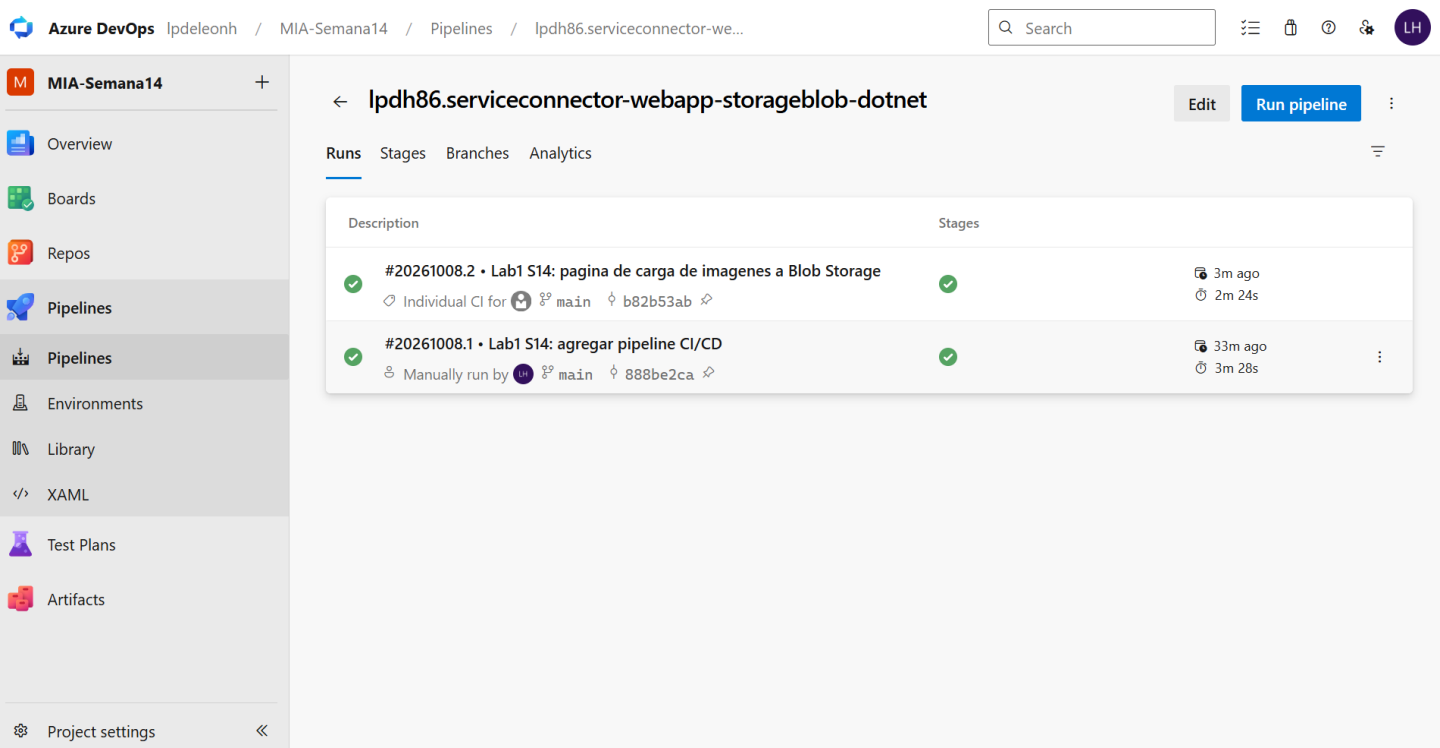


Figura 23. Paso 8 (conectar pipeline con la app): ejecuciones del pipeline; la segunda se inició automáticamente por el commit en main

9. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

a) acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings -> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT
https://tu_storage.blob.core.windows.net/
(si existe obviar)
b) Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica
c) En la app publicada realizar cambios en archivos
d) Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios

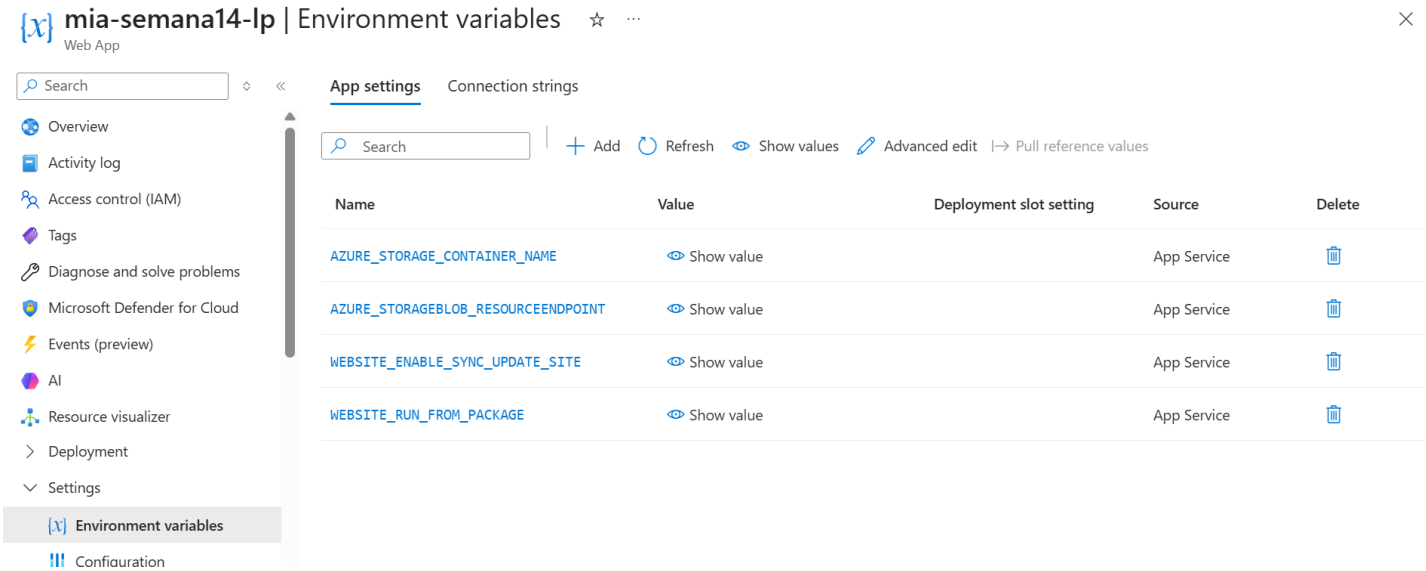


Figura 24. Paso 9a: variables de entorno del App Service (AZURE_STORAGEBLOB_RESOURCEENDPOINT y AZURE_STORAGE_CONTAINER_NAME)

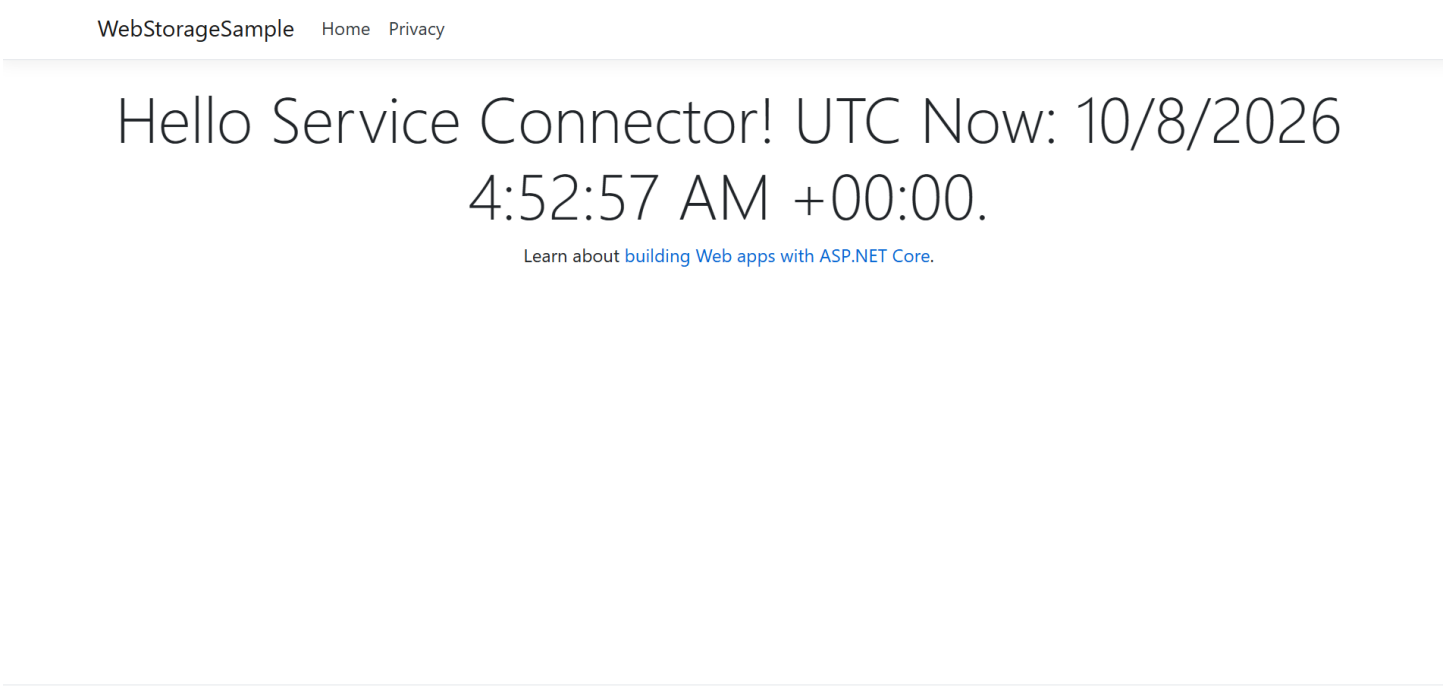


Figura 25. Paso 9b: aplicación desplegada en Azure App Service mostrando "Hello Service Connector"

---

10. Preguntas de analisis

1. ¿Para qué sirve el archivo .env en la aplicación?
   Guarda la configuración que cambia según el entorno, sin escribirla dentro del código. En este laboratorio contiene AZURE_STORAGEBLOB_RESOURCEENDPOINT y AZURE_STORAGE_CONTAINER_NAME. Con el paquete DotNetEnv, el Program.cs lo carga al iniciar y esos valores quedan como variables de entorno que la app lee con Environment.GetEnvironmentVariable. Hay un .env para local y un .env.production, que el pipeline copia a .env antes de compilar. Así el mismo código funciona en ambos entornos. Estos archivos solo tienen el endpoint y el nombre del contenedor, no claves ni contraseñas.
2. ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?
   Los archivos dentro del proyecto viajan con cada despliegue y se pueden perder o sobrescribir cuando se publica una versión nueva.
   El sistema de archivos del App Service no está pensado para guardar datos de usuarios de forma duradera.
   Blob Storage está hecho para esto: tiene capacidad casi ilimitada, redundancia, permisos propios y se paga por uso.
   Separa los datos del código. Puedes cambiar o redesplegar la app sin tocar los archivos, y varias instancias de la app comparten el mismo almacenamiento.
3. ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?
   El endpoint de Blob Storage (https://miarchivos1227025.blob.core.windows.net/) es la dirección del servicio de almacenamiento. La app la usa como cliente para leer y escribir blobs, y requiere autenticación porque el contenedor es privado.
   La URL de App Service (https://mia-semana14-lp....azurewebsites.net) es la dirección pública de la aplicación web, la que abren los usuarios en el navegador.
4. ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?
   La key da acceso total a toda la cuenta, y si se filtra en el código, en un repositorio o en un log, cualquiera puede usarla.
   Hay que guardarla, rotarla y distribuirla manualmente.
   La identidad administrada la crea y la gestiona Azure, y no hay contraseña que guardar ni filtrar.

Los permisos se asignan con RBAC, con el mínimo necesario: aquí Storage Blob Data Contributor, que solo da acceso a los datos de blobs.
Se puede revocar en cualquier momento y queda registro de quién accedió.

5. ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?
   GitHub: guarda el código fuente y el historial de cambios (control de versiones).
   Azure DevOps (Pipelines): ejecuta el pipeline definido en azure-pipelines.yml, que restaura, compila, publica y despliega la aplicación automáticamente.
   Service Connection (MIA_Connection): es la autorización que permite a Azure DevOps entrar a mi suscripción de Azure para desplegar en el App Service, sin guardar credenciales en el YAML. Usa Workload Identity Federation.
6. ¿Qué ocurre después de hacer push a main?
   GitHub recibe el commit y Azure DevOps detecta el cambio, porque el pipeline tiene trigger: main.
   Se inicia un agente windows-latest, que descarga el código.
   Instala el SDK de .NET 9 y restaura las dependencias.
   Copia .env.production sobre .env.
   Compila en modo Release y publica la aplicación en un .zip.
   Usa la service connection para desplegar el paquete en mia-semana14-lp.
   La nueva versión queda disponible en la URL pública sin intervención manual.
7. Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?
   La identidad administrada del App Service: que System assigned esté en On.
   El rol: en el Storage Account, en Access control (IAM) → Role assignments, que la identidad de mia-semana14-lp tenga Storage Blob Data Contributor y no solo Reader, y que esté asignado en el Storage correcto.
   El tiempo de propagación: los roles pueden tardar unos minutos en aplicarse.
   La configuración: que AZURE_STORAGEBLOB_RESOURCEENDPOINT apunte al Storage correcto, con la barra final, y que el nombre del contenedor coincida.
   Las restricciones de red del Storage: que el acceso público de red o el firewall no bloqueen al App Service.
   Los logs: el Log stream del App Service y el código de error. Yo vi AuthorizationPermissionMismatch en local, que indica justamente que faltaba el rol para ese usuario.